

Report - Version 3: Single Hidden Layer Network

Dataset	Task	Architecture	Test acc.
Fashion-MNIST	10 classes	784 -> 64 -> 10	79.02%

Introduction

Version 3 extends the multiclass linear model by adding one hidden layer with 64 neurons. The goal is to introduce non-linearity while keeping the implementation compact and fully transparent.

The network is still built from scratch with NumPy. It now performs a two-layer forward pass and propagates the error backward through both layers.

Goal of Version 3

The main goal is to understand why hidden layers are useful and how backpropagation distributes the error through multiple sets of weights and biases.

Data and Preparation

The project uses the Fashion-MNIST CSV dataset with 60,000 labeled training images and 10,000 labeled test images. Each 28 x 28 grayscale image is represented by 784 pixel values.

All 10 classes are used. Pixel values are normalized from 0-255 to 0-1, and integer labels are converted to one-hot vectors for multiclass cross-entropy.

Architecture

The flow of Version 3 is:

784 pixels -> 64 hidden neurons -> ReLU -> 10 output scores -> softmax -> argmax

Hidden Layer and ReLU

The first weight matrix has shape 784 x 64 and maps each image to 64 hidden activations. ReLU keeps positive values and sets negative values to zero, allowing the network to learn non-linear representations.

Initialization

The two weight matrices use a He-style random initialization. A fixed random seed (42) makes the experiment reproducible across runs.

Softmax and Cross-Entropy

The second layer maps the 64 hidden activations to 10 class scores. Softmax converts these scores into class probabilities, and multiclass cross-entropy measures the prediction error.

Backpropagation

The output error is propagated backward through the second layer, through the ReLU derivative, and then through the first layer. This produces gradients for w_2 , b_2 , w_1 , and b_1 .

Training

Training uses full-batch gradient descent with learning rate 0.1 for 200 fixed epochs. A fixed epoch count keeps execution predictable and results reproducible.

Training Result

After 200 epochs, the final training loss is 0.5895 and training accuracy is 79.03%.

Evaluation

The test set follows the same normalization and two-layer forward pass. The class with the highest softmax probability is selected with argmax. Test accuracy is 79.02%.

Results

Metric	Value
Training loss	0.5895
Training accuracy	79.03%
Test accuracy	79.02%

What Version 3 Teaches

Version 3 introduces the first genuinely non-linear network in the project. The hidden layer can transform the original pixels before classification, while backpropagation makes it possible to train both layers together.

The version is intentionally simple: its purpose is to make hidden activations, ReLU, parameter initialization, and two-layer backpropagation visible rather than to maximize accuracy.

Current Performance

With this exact educational configuration, the network reaches 79.03% training accuracy and 79.02% test accuracy. Adding representational capacity alone does not automatically improve results; training strategy and architecture must evolve together.

Limitations

Version 3 still uses full-batch gradient descent, a fixed learning rate, one hidden layer, and no regularization or advanced optimizer. Training is therefore computationally heavier than the linear model while remaining deliberately easy to inspect.

Conclusion

Version 3 successfully adds a 64-neuron hidden layer, ReLU activation, reproducible initialization, and backpropagation through two layers. The network remains compact and from scratch, while establishing the core architecture needed for later improvements.